

# SEGUNDO PARCIAL

## Programación Móvil

*Kotlin + Jetpack Compose + MVVM + Retrofit + Hilt*

**Estudiante: Miguel Angel Montoya**

---

# 1. Introducción

El presente informe corresponde al Segundo Parcial del módulo de Programación Móvil. En él se documentan dos proyectos Android desarrollados con tecnologías modernas: Kotlin, Jetpack Compose, arquitectura MVVM, Room, Retrofit y Hilt.

El trabajo integra el uso de Inteligencia Artificial generativa como herramienta de apoyo en el diseño de interfaces, generación de código base y resolución de errores de compilación, siguiendo la política pedagógica del curso.

## 2. Proyecto 1 — App Multi-Taller (5 Talleres)

Aplicación Android integral construida progresivamente en cinco talleres, con menú principal de navegación tipo dashboard y módulos independientes para cada taller.

### 2.1 Descripción General

**Nombre del proyecto:** Parcial02 — App Multi-Taller

**Arquitectura:** MVVM con Clean Architecture

**UI:** Jetpack Compose + Material3

**Persistencia:** Room Database

**Red:** Retrofit + OkHttp

**Inyección de dependencias:** Hilt

### 2.2 Tecnologías Utilizadas

| Tecnología            | Uso en el Proyecto                                                          |
|-----------------------|-----------------------------------------------------------------------------|
| Jetpack Compose       | Construcción de toda la UI de forma declarativa, eliminando XML de layout   |
| Material3             | Sistema de diseño moderno con soporte para dark mode y Material You         |
| Navigation Compose    | NavHost + NavController para navegación entre pantallas                     |
| ViewModel + StateFlow | Gestión de estado reactivo en arquitectura MVVM                             |
| Room Database         | Persistencia local de tareas, transacciones financieras y recetas favoritas |
| Retrofit + Gson       | Consumo de APIs REST (OpenWeatherMap, Spoonacular)                          |
| OkHttp + Logging      | Cliente HTTP con interceptor de logs para debugging                         |
| Hilt                  | Inyección de dependencias con integración directa al ciclo de vida Android  |

|                   |                                                       |
|-------------------|-------------------------------------------------------|
| <b>Coil</b>       | Carga de imágenes desde URLs en Composables           |
| <b>Coroutines</b> | Operaciones asíncronas sin bloquear el hilo principal |

## 2.3 Estructura de Paquetes

|                            |                                                                          |
|----------------------------|--------------------------------------------------------------------------|
| app/src/main/java/com.app/ |                                                                          |
| ├─ navigation/             | → AppNavGraph.kt, Routes.kt                                              |
| ├─ ui/                     |                                                                          |
| │   └─ screens/            | → WelcomeScreen, TaskScreen, WeatherScreen, FinanceScreen, RecipesScreen |
| │   └─ components/         | → Componentes reutilizables (Cards, Chips, etc.)                         |
| ├─ data/                   |                                                                          |
| │   └─ local/              | → AppDatabase, Entities, DAOs                                            |
| │   └─ remote/             | → API Services, Retrofit instances                                       |
| │   └─ repository/         | → Implementaciones de repositorios                                       |
| ├─ domain/                 |                                                                          |
| │   └─ model/              | → Modelos de negocio puros                                               |
| │   └─ usecase/            | → Casos de uso                                                           |
| ├─ viewmodel/              | → ViewModels por módulo                                                  |
| └─ di/                     | → Módulos Hilt                                                           |

## 2.4 Prompt General de Contexto (IA)

### Prompt de Contexto General — Enviado al inicio de cada sesión

Actúa como un desarrollador senior Android especializado en Kotlin, Jetpack Compose, Material3 y arquitectura MVVM.

Tu trabajo será:

- corregir errores de compilación
- explicar errores
- generar código funcional
- usar buenas prácticas Android modernas
- mantener arquitectura limpia
- usar Material3
- explicar cada archivo generado

IMPORTANTE:

- No generes pseudocódigo. Todo debe compilar.
- Explica imports y dependencias.
- Si usas APIs experimentales agrega automáticamente @OptIn.
- Usa nombres claros. Mantén código organizado por paquetes.

## 2.5 Taller 1 — Hola Compose: Pantalla de Bienvenida

### Tema: UI Declarativa con Jetpack Compose

#### Objetivo

Construir la primera pantalla declarativa en Jetpack Compose, con tarjeta de perfil, uso correcto de Composables básicos y @Preview funcional.

#### Requisitos implementados

- Pantalla de bienvenida con Surface, Column, Row y Text
- Tarjeta de perfil con imagen circular (painterResource)
- @Preview funcional visible en Android Studio
- Material3 con MaterialTheme.colorScheme y shapes
- Modifier correctamente encadenado (fillMaxWidth, padding, clip)

#### Prompt utilizado para el Taller 1

##### Prompt Taller 1

Ahora implementa el Taller 1 del PDF. Necesito:

- pantalla de bienvenida declarativa
- tarjeta de perfil
- uso de Text, Column, Row, Image
- @Preview funcional
- Material3 moderno
- Surface y Modifier correctamente usados

Además explica:

- la diferencia entre XML y Compose
- qué es recomposition
- qué es remember
- corrige cualquier error de compilación

Quiero: 1. Código completo 2. Imports completos  
3. Explicación archivo por archivo 4. Buenas prácticas

#### Conceptos clave aprendidos

- @Composable: transforma funciones Kotlin en elementos de UI
- Recomposition: re-ejecución selectiva cuando el estado cambia
- remember: conserva valores entre recomposiciones
- Modifier: cadena de ajustes de tamaño, padding, color y click
- XML vs Compose: Compose elimina inflate, findViewById y setters

#### Rúbrica de Evaluación

| Criterio             | Descripción                                         | Puntos |
|----------------------|-----------------------------------------------------|--------|
| Proyecto configurado | Compose corre sin errores en emulador               | 20     |
| Composables básicos  | Uso correcto de Text, Column, Row, Modifier         | 30     |
| @Preview funcional   | Previsualización visible en Android Studio          | 20     |
| Actividad IA         | Análisis crítico del código generado y modificación | 30     |

## 2.6 Taller 2 — Estado y Navegación: App de Lista de Tareas

### Tema: Estado Reactivo + NavController

#### Objetivo

Implementar gestión de estado con remember/mutableStateOf, lista dinámica con LazyColumn y navegación entre pantallas con NavHost y NavController.

#### Requisitos implementados

- Menú principal tipo dashboard con cards por cada taller
- NavHost con rutas definidas para los 5 talleres
- LazyColumn con keys únicas para renderizado eficiente
- TextField con FloatingActionButton para agregar tareas
- SwipeToDismiss con @OptIn(ExperimentalMaterial3Api::class)
- Pantalla de detalle de tarea con navegación y regreso
- Animaciones suaves de navegación entre pantallas

#### Prompt utilizado para el Taller 2

##### Prompt Taller 2

Ahora implementa el Taller 2, pero además quiero una estructura general de aplicación con navegación principal entre todos los talleres.

REQUERIMIENTO NUEVO IMPORTANTE:

Crea un menú principal moderno con NavHost y NavController.

El menú debe tener: Cards para cada taller, Material3 limpio, pantalla tipo dashboard, íconos representativos, animaciones suaves, Scaffold con TopAppBar.

Opciones del menú:

- Taller 1 → Pantalla de bienvenida
- Taller 2 → Lista de tareas
- Taller 3 → App clima
- Taller 4 → Finanzas personales
- Taller 5 → Recetas IA

Ahora implementa Taller 2 con: LazyColumn, navegación Compose, estado reactivo, remember y mutableStateOf, TextField para tareas, FloatingActionButton, SwipeToDismiss, animaciones, pantalla detalle.

IMPORTANTE: Si usas SwipeToDismiss agrega @OptIn(ExperimentalMaterial3Api::class)

## Lo que aprendí del código generado por IA

- SwipeToDismiss requiere anotación @OptIn para APIs experimentales de Material3
- LazyColumn acepta key = { it.id } para optimizar recomposiciones
- remember { mutableStateOf() } vs rememberSaveable: el segundo sobrevive rotaciones
- NavController.navigate() acepta argumentos como parte de la ruta

## Rúbrica de Evaluación

| Criterio              | Descripción                                    | Puntos |
|-----------------------|------------------------------------------------|--------|
| Estado reactivo       | Lista se actualiza al agregar/completar tareas | 25     |
| LazyColumn funcional  | Renderizado eficiente con keys únicas          | 20     |
| Navegación            | Flujo lista → detalle → regreso correcto       | 25     |
| Validación de entrada | TextField con mensajes de error visibles       | 15     |
| Actividad IA          | Componente generado, integrado y documentado   | 15     |

## 2.7 Taller 3 — ViewModel + StateFlow: App del Clima

### Tema: MVVM + Retrofit + OpenWeatherMap API

#### Objetivo

Implementar arquitectura MVVM completa con ViewModel, StateFlow y consumo de la API de OpenWeatherMap usando Retrofit, con estados Loading, Success y Error.

#### Requisitos implementados

- WeatherUiState como sealed class (Loading, Success, Error)
- WeatherViewModel con MutableStateFlow y viewModelScope.launch
- WeatherRepository con función suspendida getWeather(city)
- Retrofit configurado con OkHttpClient y LoggingInterceptor
- Temperatura grande centrada con ícono animado del clima
- Chips para humedad y viento con FilterChip de Material3
- Fondo dinámico con gradiente usando animateColorAsState
- CircularProgressIndicator para estado de carga
- Mensajes de error amigables con Snackbar o ErrorMessage Composable

#### Prompt utilizado para el Taller 3

##### Prompt Taller 3

Ahora implementa el Taller 3. Mantén el menú principal y agrega acceso al Taller 3 desde el dashboard.

Quiero una app de clima moderna con MVVM. REQUISITOS:

- Temperatura grande centrada
- Ícono animado del clima
- Chips para humedad y viento
- Fondo dinámico: Soleado=naranja, Lluvia=azul, Noche=morado
- Uso de animateColorAsState
- Diseño Material You moderno y accesible

ARQUITECTURA: MVVM + ViewModel + StateFlow + Repository + UI State

CONSUMO API: Retrofit + OpenWeatherMap + Loading/Success/Error

ANIMACIONES: animateColorAsState + AnimatedVisibility

Explica: cómo funciona animateColorAsState, Retrofit, StateFlow/MVVM.  
Evalúa si la UI es usable, accesible y sigue Material You.

#### Arquitectura MVVM implementada

```
// WeatherUiState.kt
sealed class WeatherUiState {
```



```

    object Loading : WeatherUiState()
    data class Success(val data: WeatherData) : WeatherUiState()
    data class Error(val message: String) : WeatherUiState()
}

// WeatherViewModel.kt
class WeatherViewModel(private val repository: WeatherRepository) : ViewModel() {
    private val _uiState = MutableStateFlow<WeatherUiState>(WeatherUiState.Loading)
    val uiState: StateFlow<WeatherUiState> = _uiState.asStateFlow()
}

```

## Rúbrica de Evaluación

| Criterio                | Descripción                                     | Puntos    |
|-------------------------|-------------------------------------------------|-----------|
| <b>MVVM correcto</b>    | Separación clara VM / Repository / UI           | <b>25</b> |
| <b>Llamada API real</b> | Datos de clima reales visualizados              | <b>20</b> |
| <b>Estados de UI</b>    | Loading, Success y Error visibles y funcionales | <b>25</b> |
| <b>Diseño UI IA</b>     | Pantalla del clima diseñada con apoyo de IA     | <b>20</b> |
| <b>Extensión Room</b>   | Historial de ciudades persiste entre sesiones   | <b>10</b> |

## 2.8 Taller 4 — Room + Hilt: App de Finanzas Personales

### Tema: Room + Hilt + Clean Architecture

#### Objetivo

Implementar persistencia local con Room Database, inyección de dependencias con Hilt y módulo completo de finanzas personales con CRUD de transacciones.

#### Requisitos implementados

- TransaccionEntity con @Entity y @PrimaryKey(autoGenerate=true)
- TransaccionDao con @Query, @Insert, @Delete y Flow<List<>>
- AppDatabase con @Database y singleton pattern
- FinanzasRepository con implementación concreta
- AgregarTransaccionUseCase con validación de campos
- FinanzasViewModel con @HiltViewModel e @Inject constructor
- Pantalla dashboard con total ingresos, gastos y balance
- Filtros: Todos, Ingresos, Gastos, Por categoría
- Búsqueda de movimientos financieros
- Categorías: Comida, Transporte, Estudio, Entretenimiento, Salud, Otros
- Mensajes amigables: Snackbar para confirmación y errores

#### Prompt utilizado para el Taller 4

##### Prompt Taller 4

Necesito desarrollar el TALLER 4: FINANZAS PERSONALES.

##### REQUERIMIENTOS:

- Registrar ingresos y gastos con título, monto, categoría, tipo y fecha
- Eliminar movimientos
- Visualizar balance total, total ingresos y total gastos
- Categorías: Comida, Transporte, Estudio, Entretenimiento, Salud, Otros
- Filtros: Todos, Ingresos, Gastos, Por categoría
- Búsqueda de movimientos
- Persistencia con Room Database
- Entity, DAO, Database, Repository, ViewModel con Hilt
- Arquitectura MVVM
- Validaciones: no campos vacíos, no montos negativos
- NO romper: módulo de tareas, clima, pantalla de bienvenida

#### Esquema de Base de Datos Room

```
@Entity(tableName = "transacciones")
data class TransaccionEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
```

```
val descripcion: String,
val monto: Double,
val tipo: String,          // "INGRESO" | "GASTO"
val categoria: String,
val fecha: Long = System.currentTimeMillis()
)
```

Rúbrica de Evaluación

| Criterio             | Descripción                                        | Puntos |
|----------------------|----------------------------------------------------|--------|
| Room funcional       | CRUD de transacciones persiste entre sesiones      | 20     |
| Hilt configurado     | Inyección correcta en ViewModel, sin getInstance() | 20     |
| Clean Architecture   | Separación de capas verificable en paquetes        | 20     |
| Gráfica de datos     | Visualización de ingresos vs gastos funcional      | 20     |
| Sistema de diseño IA | Theme aplicado consistentemente en la app          | 20     |

## 2.9 Taller 5 — App Completa: Recetas de Cocina con IA

Tema: API REST + Room + CameraX + ML Kit

### Objetivo

Desarrollar el módulo final de Recetas de Cocina, integrando consumo de API pública (Spoonacular), Room para favoritos, Coil para imágenes y arquitectura MVVM completa.

### Requisitos implementados

- RecipeApiService con Retrofit para consumir API de recetas
- RecipeRepository con fuentes de datos remotas y locales
- RecipeViewModel con StateFlow y manejo de Loading/Success/Error
- Pantalla de lista con buscador, filtros por categoría y cards estilizadas
- Carga de imágenes con Coil (AsyncImage)
- Pantalla de detalle con imagen grande, ingredientes y pasos
- FavoriteEntity en Room para guardar recetas sin conexión
- FavoriteDao con @Insert, @Delete y Flow<List<FavoriteEntity>>
- Botón de favoritos animado con animateFloatAsState
- Mensajes: 'Receta guardada', 'No se encontraron recetas', 'Error al cargar'
- Compatibilidad total con módulos anteriores (tareas, clima, finanzas)

### Prompt utilizado para el Taller 5

#### Prompt Taller 5

Necesito desarrollar el TALLER 5: RECETAS DE COCINA.

La app YA tiene: pantalla de bienvenida, CRUD tareas, clima con OpenWeather, finanzas personales, Navigation Compose, MVVM, Room, Retrofit, Material3.

REQUERIMIENTOS:

- Ver, buscar, filtrar recetas y ver detalles completos
- Guardar recetas favoritas (Room)
- Consumir API pública de recetas con Retrofit
- Pantalla de detalle: imagen, ingredientes, instrucciones, favoritos
- Imágenes con Coil
- NO romper funcionalidades anteriores

Genera TODOS los archivos: build.gradle.kts, Retrofit, API Service, Models, Repository, Room, DAO, Entity, ViewModel, Screens, Navigation

### Rúbrica Final del Proyecto

| Criterio                 | Descripción                                        | Puntos |
|--------------------------|----------------------------------------------------|--------|
| API de recetas funcional | Datos reales cargados desde API con Retrofit       | 20     |
| Favoritos con Room       | Recetas guardadas persisten entre sesiones         | 20     |
| Arquitectura completa    | MVVM + Clean + Hilt + Room correctamente aplicados | 20     |
| Animaciones / UX         | Transiciones, feedback visual fluido               | 15     |
| Diseño con IA            | UI diseñada con apoyo de IA y personalizada        | 15     |
| Integración con menú     | Navegación integrada con el dashboard principal    | 10     |

## 3. Proyecto 2 — App de Noticias (Segundo Parcial Parte 2)

Aplicación Android de noticias construida con Jetpack Compose, Retrofit, MVVM y Hilt, que consume la API de NewsAPI.org para mostrar titulares en tiempo real.

### 3.1 Descripción General

**Nombre del proyecto:** PARCIAL02PARTE2 — NewsApp

**API:** NewsAPI.org (top-headlines)

**Arquitectura:** MVVM con Hilt

**UI:** Jetpack Compose + Material3

**Persistencia:** Room Database (caché de noticias)

**Red:** Retrofit + OkHttp + Logging Interceptor

### 3.2 Tecnologías Utilizadas

| Tecnología                  | Uso en el Proyecto                                                  |
|-----------------------------|---------------------------------------------------------------------|
| Jetpack Compose             | UI declarativa con LazyColumn para lista de noticias                |
| Hilt                        | Inyección de dependencias para NewsProvider, Repository y ViewModel |
| Retrofit + Gson             | Consumo de NewsAPI REST con GsonConverterFactory                    |
| OkHttp + LoggingInterceptor | Cliente HTTP con trazas de peticiones para debugging                |
| Room Database               | Caché local de noticias para modo offline                           |
| Coil                        | Carga de imágenes de noticias con AsyncImage y placeholder          |
| Navigation Compose          | Navegación de lista a pantalla de detalle de noticia                |
| Coroutines + StateFlow      | Gestión asíncrona y estado reactivo en ViewModel                    |

### 3.3 Estructura del Proyecto

```
app/src/main/java/com.raulsaurio.newapp/
├── di/
│   ├── ProviderModule.kt    → @Provides Retrofit + NewsProvider
│   └── RepositoryModule.kt  → @Binds NewsRepository → NewsRepositoryImp
├── model/
│   ├── News.kt              → data class con title, content, author, url, urlToImage
│   └── NewsApiResponse.kt    → status, code, articles: List<News>
├── provider/
│   └── NewsProvider.kt       → interface con @GET top-headlines
├── repository/
│   └── NewsRepository.kt     → interface + NewsRepositoryImp
├── ui/
│   ├── ListScreen.kt        → LazyColumn de noticias
│   └── DetailsScreen.kt     → Detalle con imagen grande y botón Ver más
```

## 3.4 Arquitectura de la App

La aplicación sigue el flujo de capas recomendado por Google: la Data Layer contiene SQLite (Room) y el NewsProvider (Retrofit). El NewsRepository centraliza las fuentes de datos. Los ViewModels exponen el estado a las pantallas Compose.

## 3.5 Configuración de Dependencias (build.gradle.kts)

```
// build.gradle.kts (Project)
id("com.google.dagger.hilt.android") version "2.51.1" apply false
id("com.google.devtools.ksp") version "1.9.23-1.0.20" apply false

// build.gradle.kts (Module: app)
implementation("io.coil-kt.coil3:coil-compose:3.3.0")
implementation("androidx.navigation:navigation-compose:2.9.5")
implementation("com.squareup.retrofit2:retrofit:3.0.0")
implementation("com.squareup.retrofit2:converter-gson:3.0.0")
implementation("com.squareup.okhttp3:logging-interceptor:5.1.0")
implementation("androidx.hilt:hilt-navigation-compose:1.3.0")
implementation("com.google.dagger:hilt-android:2.51.1")
ksp("com.google.dagger:hilt-compiler:2.51.1")
// Test con Hilt
androidTestImplementation("com.google.dagger:hilt-android-testing:2.51.1")
kspAndroidTest("com.google.dagger:hilt-compiler:2.51.1")
// Test con Retrofit (MockWebServer)
testImplementation("com.squareup.okhttp3:mockwebserver:5.1.0")
```

## 3.6 Prompt utilizado para corregir error HTTP 401

### Prompt Corrección NewsAPI 401

Estoy desarrollando una app Android con Kotlin + Jetpack Compose + MVVM + Retrofit + Hilt para consumir noticias desde una API REST.

La aplicación actualmente muestra: HTTP 401 Unauthorized

#### REQUERIMIENTOS:

1. Verificar que la API KEY esté correctamente implementada
2. Verificar que NO existan: espacios extra, saltos de línea, caracteres invisibles
3. Verificar que la Base URL sea: `https://newsapi.org/v2/`
4. Verificar endpoint: `top-headlines?country=us&apiKey=API_KEY`
5. Corregir completamente Retrofit: Builder, GsonConverterFactory, OkHttpClient
6. Verificar AndroidManifest: `<uses-permission INTERNET/>`
7. Manejar estados: loading, success, error
8. Mensajes amigables: 'La API aún no está activa', 'API Key inválida'
9. Si NewsAPI sigue dando 401, reemplazar por API gratuita alternativa

## 3.7 Modelos de Datos

```
// News.kt
data class News(
    var title: String,
    var content: String?,
    var author: String?,
    var url: String,
    var urlToImage: String
)

// NewsApiResponse.kt
class NewsApiResponse {
    val status: String? = null
    val code: String? = null
    val articles: List<News>? = null
}
```

## 3.8 Interfaz del Provider con Retrofit

```
// NewsProvider.kt
private const val API_KEY = "YOUR_API_KEY_HERE"

interface NewsProvider {
    @GET("top-headlines?apiKey=$API_KEY")
    suspend fun topHeadLines(
        @Query("country") country: String
    ): Response<NewsApiResponse>
}
```

## 3.9 Pruebas Unitarias

### Conceptos aplicados

- JUnit para validar lógica de negocio de forma aislada
- MockWebServer (OkHttp) para simular respuestas de la API sin conexión real
- Hilt Android Testing para inyección de dependencias en tests
- coroutines-test para validar funciones suspendidas y flujos reactivos

```
// Ejemplo de test con MockWebServer
@Test
fun `cuando la API responde 200 el ViewModel emite Success`() = runTest {
    mockWebServer.enqueue(MockResponse()
        .setResponseCode(200)
        .setBody(/* JSON de ejemplo */))
    viewModel.loadNews("us")
    val state = viewModel.uiState.first { it !is Loading }
    assertTrue(state is NewsUiState.Success)
}
```



## 4. Lo que Aprendí del Código Generado por IA

A lo largo de los dos proyectos se utilizó IA generativa (GEMINI, CHATGPT) como asistente de desarrollo. A continuación se documentan los aprendizajes más relevantes.

### 4.1 Fortalezas del uso de IA

- Generación rápida de boilerplate: Entity, DAO, Repository, ViewModel con Hilt
- Corrección inmediata de errores de compilación con explicación del por qué
- Sugerencia automática de `@OptIn` cuando se usan APIs experimentales de Material3
- Generación de código MVVM bien estructurado con separación de responsabilidades
- Explicaciones de conceptos como `animateColorAsState`, `StateFlow` y `Coroutines`

### 4.2 Limitaciones encontradas

- La IA a veces sugería versiones desactualizadas de dependencias (ej. Coil 2.x en lugar de 3.x)
- Código de UI generado necesitaba ajustes manuales de padding, colores y espaciado
- En arquitectura Clean, la IA tendía a mezclar responsabilidades entre capas
- Los imports no siempre eran correctos al primer intento; requerían revisión
- El error HTTP 401 de NewsAPI requirió análisis propio para identificar el problema real

### 4.3 Conclusión

La IA es una herramienta poderosa para acelerar el desarrollo Android, especialmente en la generación de código repetitivo y la corrección de errores de compilación. Sin embargo, el desarrollador debe comprender el código generado, verificar las dependencias y adaptar el resultado a las necesidades específicas del proyecto. El pensamiento crítico sigue siendo indispensable.

---

## 5. Conclusiones

- Se implementaron con éxito dos proyectos Android usando el stack moderno: Kotlin, Jetpack Compose, MVVM, Room, Retrofit y Hilt.
- La arquitectura MVVM con Clean Architecture demostró su valor al mantener los módulos independientes y sin romper funcionalidades entre talleres.
- Room Database garantizó la persistencia local de tareas, transacciones financieras, historial del clima y recetas favoritas.
- Hilt simplificó significativamente la gestión de dependencias, especialmente en la inyección de Repository en ViewModel.
- El uso de StateFlow con estados sellados (sealed class) permitió manejar Loading, Success y Error de forma elegante y predecible.
- La IA generativa fue un apoyo valioso pero no un sustituto del análisis crítico del desarrollador.

